Zaawansowane algorytmy wizyjne

Skrypt do ćwiczeń laboratoryjnych

Tomasz Kryjak, Mateusz Wąsala, Hubert Szolc, Piotr Wzorek

Copyright © 2026
Tomasz Kryjak
Mateusz Wąsala
Hubert Szolc
Piotr Wzorek
Krzysztof Błachut

Zespół Wbudowanych Systemów Wizyjnych
Laboratorium Systemów Wizyjnych
WEAIiIB, AGH w Krakowie

Wydanie czwarte, zmienione i poprawione
Kraków, luty 2026

Spis treści

I Blok 1

1 Laboratorium 1

1 Algorytmy wizyjne w Python 3.X – wstęp 9
1.1 Wykorzystywane oprogramowanie 9
1.2 Moduły Pythona wykorzystywane w przetwarzaniu obrazów 9
1.3 Operacje wejścia/wyjścia 10
1.4 Konwersje przestrzeni barw 11
1.4.1 OpenCV 11
1.4.2 Matplotlib 11
1.5 Skalowanie, zmiana rozdzielczości przy użyciu OpenCV 12
1.6 Operacje arytmetyczne: dodawanie, odejmowanie, mnożenie, moduł z różnicy 12
1.7 Wyliczenie histogramu 13
1.8 Wyrównywanie histogramu 13
1.9 Filtracja 14

Blok 1

1 Laboratorium 1

1 Algorytmy wizyjne w Python 3.X – wstęp 9

1.1 Wykorzystywane oprogramowanie
1.2 Moduły Pythona wykorzystywane w przetwarzaniu obrazów
1.3 Operacje wejścia/wyjścia
1.4 Konwersje przestrzeni barw
1.5 Skalowanie, zmiana rozdzielczości przy użyciu OpenCV
1.6 Operacje arytmetyczne: dodawanie, odejmowanie, mnożenie, moduł z różnicy
1.7 Wyliczenie histogramu
1.8 Wyrównywanie histogramu
1.9 Filtracja

1. Algorytmy wizyjne w Python 3.X – wstęp

W ramach ćwiczenia zaprezentowane/przypomniane zostaną podstawowe informacje związane z wykorzystaniem języka Python do realizacji operacji przetwarzania i analizy obrazów, a także sekwencji wideo.

1.1 Wykorzystywane oprogramowanie

Przed rozpoczęciem ćwiczeń **proszę utworzyć** własny katalog roboczy w miejscu podanym przez Prowadzącego. Nazwa katalogu powinna być związana z Państwa nazwiskiem – zalecana forma to `NazwiskoImie` bez polskich znaków.

Do przeprowadzania ćwiczeń można wykorzystać jedno z trzech środowisk programowania w Pythonie – Spyder5, PyCharm lub Visual Studio Code (VSCode). Ich zaletą jest możliwość łatwego podglądu tablic dwuwymiarowych (czyli obrazów). Spyder jest prostszy, ale także ma mniejsze możliwości i więcej niedociągnięć. PyCharm jest oprogramowaniem komercyjnym udostępnianym także w wersji darmowej (wygląd zbliżony do CLion dla C/C++). W PyCharmie pracę należy zacząć od utworzenia projektu, Spyder pozwala na pracę na pojedynczych plikach (bez konieczności tworzenia projektu).

Visual Studio Code jest darmowym, lekkim, intuicyjnym i łatwym w obsłudze edytorem kodu źródłowego. Jest to uniwersalne oprogramowanie do dowolnego języka programistycznego. Konfiguracja programu do ćwiczeń sprowadza się jedynie do wybrania odpowiedniego interpretera, a dokładnie odpowiedniego środowiska wirtualnego w języku Python.

1.2 Moduły Pythona wykorzystywane w przetwarzaniu obrazów

Python udostępnia wbudowane kontenery (np. listy), ale nie oferuje natywnego typu tablicowego zaprojektowanego do wydajnych operacji wektorowych/element–po–elemencie (np. dodawania, mnożenia czy progowania całych bloków danych). W praktyce standardem jest użycie modułu *NumPy*, który dostarcza typ `ndarray` oraz mechanizmy takie jak wektoryzacja i *broadcasting*. Obraz cyfrowy można wtedy traktować jako tablicę $H \times W$ (skala

szarości) lub $H \times W \times C$ (kolor), zwykle o typie `uint8` (zakres 0–255) albo `float` (np. 0–1). *NumPy* stanowi fundament dla większości bibliotek używanych dalej do przetwarzania i wizualizacji obrazów.

Najczęściej spotkane pakiety wspierające przetwarzanie obrazów to m.in.:

- *SciPy* (moduł *ndimage*) – podstawowe operacje filtracji i przekształceń (również wielowymiarowych) oraz *Matplotlib* (*pyplot*) – wizualizacja obrazów i wykresów,
- *Pillow* (fork starszego, nierozwijanego modułu PIL) – wygodne wczytywanie/zapisywanie i proste operacje na obrazach,
- *OpenCV* (moduł *cv2*) – rozbudowany zestaw algorytmów widzenia komputerowego (filtry, geometria, cechy, dopasowanie, kalibracja, itp.),
- *scikit-image* (*skimage*) – biblioteka z algorytmami przetwarzania obrazów zintegrowana z ekosystemem naukowym Pythona,
- *Plotly* (*plotly*) – interaktywne wykresy i wizualizacje (np. mapy cieplne, wykresy 3D, dashboardy w Jupyter/HTML),
- *Rerun* (*rerun*, *rerun.io*) – logowanie i interaktywna wizualizacja danych (np. obrazy, adnotacje, 3D/robotyka) w viewerze Rerun,
- *pandas* – narzędzia do pracy z danymi tabelarycznymi (np. wczytywanie adnotacji i metadanych z CSV/Excel, analiza wyników eksperymentów).

W tym kursie oprzemy się głównie na *OpenCV*. Równolegle będziemy używać *Matplotlib* (zwłaszcza do wyświetlania wyników; uwaga: *OpenCV* domyślnie przechowuje kolor w kolejności BGR, a *Matplotlib* oczekuje RGB) oraz sporadycznie *ndimage* – gdy dana operacja nie jest dostępna w *OpenCV* albo jest tam mniej wygodna w użyciu.

1.3 Operacje wejścia/wyjścia

Wczytywanie, wyświetlanie i zapisywanie obrazu z wykorzystaniem OpenCV oraz Matplotlib

Ćwiczenie 1.1 Wykonaj zadanie, w którym przećwiczysz obsługę plików z wykorzystaniem OpenCV oraz Matplotlib.

1. Ze strony kursu pobierz obraz *mandril.jpg* i umieść go we własnym katalogu roboczym.
2. Wczytaj, wyświetl oraz zapisz obraz z rozszerzeniem .png pod nową nazwą wykorzystując bibliotekę OpenCV oraz Matplotlib. Następnie umieść na obrazie **punkt** i **prostokąt** oraz wyświetl na ekranie.

R Wykorzystaj:

- OpenCV: `cv2.imread()`, `cv2.imshow()`, `cv2.imwrite()`, `cv2.waitKey()`, `cv2.destroyAllWindows()`, `cv2.rectangle()`.
- Matplotlib: `plt.imread()`, `plt.imsave()`, `plt.figure()`, `plt.imshow()`, `plt.axis()`, `plt.show()` oraz `matplotlib.patches.Rectangle()`.

R Funkcja `waitKey()` wyświetla obraz przez określony czas (argument 0 oznacza oczekiwanie na naciśnięcie klawisza). Niepotrzebne okno można zamknąć poleceniem `destroyWindow()` z odpowiednim parametrem, a wszystkie otwarte okna — poleceniem `destroyAllWindows()`. Dobrą praktyką jest zawsze stosowanie funkcji `cv2.destroyAllWindows()`

Rozliczenie ćwiczenia
W celu rozliczenia ćwiczenia po wykonaniu zadania zgłoś prowadzącemu zajęcia swoją gotowość. Alternatywnie możesz zgłosić gotowość po wykonaniu wszystkich zadań dotyczących omawianego tematu zajęć. Po akceptacji z jego strony umieść skrypt z rozszerzeniem `.py` lub `.ipynb` w odpowiednim miejscu w zasobach kursu na UPeL. ■

1.4 Konwersje przestrzeni barw

1.4.1 OpenCV

Do konwersji przestrzeni barw służy funkcja *cvtColor*.

```
IG = cv2.cvtColor(I, cv2.COLOR_BGR2GRAY)
IHSV = cv2.cvtColor(I, cv2.COLOR_BGR2HSV)
```

Uwaga! Proszę zauważyć, że w OpenCV odczyt jest w kolejności BGR, a nie RGB. Może to być istotne w przypadku ręcznego manipulowania pikselami. Pełna lista dostępnych konwersji wraz ze stosownymi wzorami w dokumentacji OpenCV

Ćwiczenie 1.2 Wykonaj konwersję przestrzeni barw podanego obrazu.

1. Dokonać konwersji obrazu *mandril.jpg* do odcieni szarości i przestrzeni HSV. Wynik wyświetlić.
2. Wyświetlić składowe H, S, V obrazu po konwersji.

Rozliczenie ćwiczenia
W celu rozliczenia ćwiczenia po wykonaniu zadania zgłoś prowadzącemu zajęcia swoją gotowość. Alternatywnie możesz zgłosić gotowość po wykonaniu wszystkich zadań dotyczących omawianego tematu zajęć. Po akceptacji z jego strony umieść skrypt z rozszerzeniem `.py` lub `.ipynb` w odpowiednim miejscu w zasobach kursu na UPeL. ■

Przydatna składnia:

```
IH = IHSV[:,:,0]
IS = IHSV[:,:,1]
IV = IHSV[:,:,2]
```

1.4.2 Matplotlib

Tu wybór dostępnych konwersji jest dość ograniczony.

1. RGB do odcieni szarości. Można wykorzystać rozbicie na poszczególne kanały i wzór:

$$G = 0.299 \cdot R + 0.587 \cdot G + 0.144 \cdot B \tag{1.1}$$

Całość można opakować w funkcję:

```
def rgb2gray(I):
  return 0.299*I[:,:,0] + 0.587*I[:,:,1] + 0.114*I[:,:,2]
```

Uwaga! Przy wyświetlaniu należy ustawić mapę kolorów. Inaczej obraz wyświetli się w domyślnej, która nie jest bynajmniej w odcieniach szarości: `plt.gray()`.

2. RGB do HSV.

```
import matplotlib # add at the top of the file
I_HSV = matplotlib.colors.rgb_to_hsv(I)
```

1.5 Skalowanie, zmiana rozdzielczości przy użyciu OpenCV

Ćwiczenie 1.3 Przeskaluj obraz *mandril*. Do skalowania służy funkcja `resize`.

Rozliczenie ćwiczenia
W celu rozliczenia ćwiczenia po wykonaniu zadania zgłoś prowadzącemu zajęcia swoją gotowość. Alternatywnie możesz zgłosić gotowość po wykonaniu wszystkich zadań dotyczących omawianego tematu zajęć. Po akceptacji z jego strony umieść skrypt z rozszerzeniem `.py` lub `.ipynb` w odpowiednim miejscu w zasobach kursu na UPeL. ■

Przykład użycia:

```
height, width = I.shape[:2] # retrieving elements 1 and 2, i.e. the corresponding
    height and width
scale = 1.75   # scale factor
Ix2 = cv2.resize(I,(int(scale*height),int(scale*width)))
cv2.imshow("Big Mandril",Ix2)
```

1.6 Operacje arytmetyczne: dodawanie, odejmowanie, mnożenie, moduł z różnicy

Obrazy są macierzami, a zatem operacje arytmetyczne są dość proste – tak jak w pakiecie Matlab. Należy oczywiście pamiętać o konwersji na odpowiedni typ danych. Zwykle dobrym wyborem będzie double.

Ćwiczenie 1.4 Wykonaj operacje arytmetyczne na obrazie *lena*.

1. Pobierz ze strony kursu obraz *lena*, a następnie go wczytaj za pomocą funkcji z OpenCV – dodaj ten fragment kodu do pliku, który zawiera wczytywanie obrazu *mandril*. Wykonaj konwersję do odcieni szarości. Dodaj macierze zawierające mandryla i Leny w skali szarości. Wyświetl wynik.
2. Podobnie wykonaj odjęcie i mnożenie obrazów.
3. Zaimplementuj kombinację liniową obrazów.
4. Ważną operacją jest moduł z różnicy obrazów. Można ją wykonać „ręcznie” – konwersja na odpowiedni typ, odjęcie, moduł (`abs`), konwersja na *uint8*. Alternatywa to wykorzystanie funkcji *absdiff* z OpenCV. Proszę obliczyć moduł z różnicy „szarych” wersji mandryla i Leny.

Rozliczenie ćwiczenia
W celu rozliczenia ćwiczenia po wykonaniu zadania zgłoś prowadzącemu zajęcia swoją gotowość. Alternatywnie możesz zgłosić gotowość po wykonaniu wszystkich zadań dotyczących omawianego tematu zajęć. Po akceptacji z jego strony umieść skrypt z rozszerzeniem `.py` lub `.ipynb` w odpowiednim miejscu w zasobach kursu na UPeL. ■

Uwaga! Przy wyświetleniu obraz może nie być poprawny, ponieważ jest on typu *float64* (*double*). W praktyce należy:
1. ograniczyć zakres wartości (`clip`),
2. ewentualnie przeskalować do przedziału `[0,255]`,
3. skonwertować do *uint8*.

```
img8 = (255*np.clip(img,0,1)).astype(np.uint8) # gdy wartosci sa w [0,1]
img8 = np.clip(img,0,255).astype(np.uint8) # gdy wartosci sa w [0,255]
```

1.7 Wyliczenie histogramu

Obliczanie histogramu można wykonać z wykorzystaniem funkcji `calcHist`. Zanim jednak do tego przejdziemy, przypomnijmy sobie podstawowe struktury sterowania w Pythonie: funkcje i podprogramy. Proszę samodzielnie dokończyć poniższą funkcję, która oblicza histogram obrazu w 256 odcieniach szarości:

```
def hist(img):
  h=np.zeros((256,1), np.float32) # creates and zeros single-column arrays
  height, width = img.shape[:2] # shape - we take the first 2 values
    for y in range(height):
...
return h
```

Histogram można wyświetlić, korzystając z funkcji `plt.hist` lub `plt.plot` z biblioteki *Matplotlib*.

Funkcja `calcHist` umożliwia obliczenie histogramu dla kilku obrazów (lub ich składowych), dlatego jako parametry otrzymuje tablice (np. obrazów), a nie pojedynczy obraz. Najczęściej jednak wykorzystywana jest następująca postać:

```
hist = cv2.calcHist([IG],[0],None,[256],[0,256])
# [IG] -- input image
# [0] -- for greyscale images there is only one channel
# None -- mask (you can count the histogram of a selected part of the image)
# [256] -- number of histogram bins
# [0, 256] -- the range over which the histogram is calculated
```

Proszę sprawdzić czy histogramy uzyskane obiema metodami są takie same.

1.8 Wyrównywanie histogramu

Wyrównywanie histogramu to popularna i ważna operacja przetwarzania wstępnego.

1. Wyrównywanie "klasyczne"jest metodą globalną — wykonuje się je na całym obrazie. W bibliotece OpenCV znajduje się gotowa funkcja realizująca ten typ wyrównania:

   ```
   IGE = cv2.equalizeHist(IG)
   ```

2. Wyrównywanie CLAHE (ang. *Contrast Limited Adaptive Histogram Equalization*) – metoda adaptacyjna, która poprawia warunki oświetleniowe na obrazie. W przeciwieństwie do klasycznego wyrównywania histogramu działa lokalnie: wyrównuje histogram w poszczególnych fragmentach obrazu, a nie dla całego obrazu. Metoda działa następująco:
   - podział obrazu na rozłączne (kwadratowe) bloki,
   - wyznaczenie histogramu w każdym bloku,

- wyrównanie histogramu z ograniczeniem maksymalnej „wysokości” (nadmiar jest redystrybuowany na sąsiednie przedziały),
- interpolacja wartości pikseli na podstawie histogramów dla sąsiednich bloków (zwykle z uwzględnieniem czterech najbliższych sąsiadów).

Szczegóły znajdują się na Wiki oraz w tutorialu OpenCV.

```
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
# clipLimit - maximum height of the histogram bar - values above are distributed
    among neighbours
# tileGridSize - size of a single image block (local method, operates on separate
    image blocks)
I_CLAHE = clahe.apply(IG)
```

Ćwiczenie 1.5 Uruchom i porównaj obie metody wyrównywania.

Rozliczenie ćwiczenia
W celu rozliczenia ćwiczenia po wykonaniu zadania zgłoś prowadzącemu zajęcia swoją gotowość. Alternatywnie możesz zgłosić gotowość po wykonaniu wszystkich zadań dotyczących omawianego tematu zajęć. Po akceptacji z jego strony umieść skrypt z rozszerzeniem `.py` lub `.ipynb` w odpowiednim miejscu w zasobach kursu na UPeL. ■

1.9 Filtracja

Filtracja to bardzo ważna grupa operacji na obrazach. W ramach ćwiczenia proszę uruchomić:

- filtrację Gaussa (`GaussianBlur`)
- filtrację Sobela (`Sobel`)
- Laplasjan (`Laplacian`)
- medianę (`medianBlur`)

R Pomocna będzie dokumentacja OpenCV.

Proszę zwrócić uwagę również na inne dostępne funkcje:

- filtrację bilateralną,
- filtry Gabora,
- operacje morfologiczne.

Ćwiczenie 1.6 Uruchom i porównaj wymienione metody.

Rozliczenie ćwiczenia
W celu rozliczenia ćwiczenia po wykonaniu zadania zgłoś prowadzącemu zajęcia swoją gotowość. Alternatywnie możesz zgłosić gotowość po wykonaniu wszystkich zadań dotyczących omawianego tematu zajęć. Po akceptacji z jego strony umieść skrypt z rozszerzeniem `.py` lub `.ipynb` w odpowiednim miejscu w zasobach kursu na UPeL. ■